



SAPIENZA
UNIVERSITÀ DI ROMA

Intelligent Packets: inferenza neurale nel data plane eBPF/XDP

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Engineering in Computer Science

Ettore Cantile

Matricola 2026562

Relatore

Prof. Marco Polverini

Anno Accademico 2025/2026

Tesi non ancora discussa

Intelligent Packets: inferenza neurale nel data plane eBPF/XDP
Sapienza Università di Roma

© 2026 Ettore Cantile. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: cantile.2026562@studenti.uniroma1.it

Indice

1	Fondamenti: eBPF, XDP e il paradigma Intelligent Packets	1
1.1	eBPF: una macchina virtuale sicura dentro il kernel	1
1.2	Punti di aggancio, mappe, XDP e tail call	2
1.3	Il paradigma Intelligent Packets	3
1.4	Ambiente sperimentale	4
2	Il design space: tre pipeline per l'inferenza neurale nel datapath	6
2.1	Architettura comune alle tre pipeline	6
2.2	Pipeline 1: hardcoded	7
2.3	Pipeline 2: template	9
2.4	Pipeline 3: modular	10
2.5	Dall'argmax all'azione: una mappa comune	11
2.6	Le tre pipeline a confronto	11
2.7	Risultati sperimentali comparativi	13
2.8	Colli di bottiglia trasversali del datapath	15
2.9	Il costo della flessibilità e la scelta della pipeline vincente	17
3	L'evoluzione della pipeline hardcoded	18
3.1	Un input vector generico, senza perdere prestazioni	18
3.2	AOT-literal: separare la compilazione dal deploy	20
3.3	Profondità variabile e studio larghezza contro profondità	22
3.4	Nota conclusiva sull'evoluzione	23

Capitolo 1

Fondamenti: eBPF, XDP e il paradigma Intelligent Packets

Questa tesi si colloca all'incrocio tra due idee: da un lato la possibilità di eseguire codice arbitrario, ma sicuro, direttamente dentro il kernel Linux tramite eBPF; dall'altro il paradigma *Intelligent Packets* (IPA), in cui un pacchetto porta con sé un piccolo modello di machine learning e ogni nodo della rete lo esegue per decidere l'inoltro, senza mai interpellare un controller centralizzato. Il lavoro qui descritto porta questa seconda idea dentro il data plane del kernel, usando la prima come strumento: un modello neurale compatto, quantizzato a interi a 8 bit, viene eseguito interamente dentro un programma XDP, a velocità di linea. Prima di entrare nel merito è necessario descrivere con un minimo di dettaglio la tecnologia su cui poggia tutto il resto.

1.1 eBPF: una macchina virtuale sicura dentro il kernel

eBPF (extended Berkeley Packet Filter) è un meccanismo del kernel Linux che permette di caricare ed eseguire piccoli programmi direttamente al suo interno, agganciandoli a eventi di sistema o di rete, senza scrivere un modulo kernel e senza ricompilare o riavviare nulla. L'idea, nata come evoluzione del vecchio filtro pacchetti BPF, si è trasformata negli anni in una vera e propria macchina virtuale general-purpose interna al kernel, con un proprio set di istruzioni, un proprio modello di memoria e un proprio insieme di funzioni ausiliarie (*helper*) che il programma può invocare per interagire in modo controllato con il resto del sistema.

Ciò che rende eBPF utilizzabile in produzione non è tanto la capacità di eseguire codice nel kernel – che, detta così, suonerebbe pericolosa – quanto il fatto che questo avvenga dentro una sandbox rigorosamente controllata. Prima che un programma venga caricato, un componente del kernel chiamato *verifier* lo analizza staticamente e ne rifiuta qualunque caratteristica potenzialmente pericolosa. Il verifier costruisce il grafo di controllo del programma e simula tutti i possibili percorsi di esecuzione, verificando una serie di proprietà: che non esistano cicli il cui numero di iterazioni non sia limitabile staticamente (un programma eBPF deve terminare in un numero finito e noto di passi); che ogni accesso alla memoria sia entro limiti dimostrabili; che ogni puntatore usato per un accesso sia stato prima validato; che il programma

non superi un budget massimo di complessità, espresso come numero di istruzioni analizzate lungo tutti i percorsi. Solo se tutte queste verifiche passano, il bytecode eBPF viene compilato *just-in-time* (JIT) in istruzioni macchina native e agganciato al punto di hook richiesto, eseguendo poi alla stessa velocità del codice kernel nativo.

Il verifier è quindi allo stesso tempo la garanzia di sicurezza di eBPF e il suo vincolo più stringente dal punto di vista di chi scrive il codice: come si vedrà ripetutamente in questa tesi, molte scelte implementative non sono dettate da eleganza o da prestazioni, ma dalla necessità di produrre codice che il verifier accetti. Due limiti concreti torneranno più volte nel seguito. Il primo è il divieto di cicli non limitati, che obbliga a *srotolare* a mano ogni iterazione il cui conteggio non sia noto a compile time – un prodotto scalare tra un vettore e dei pesi, ad esempio, va scritto come una sequenza esplicita di moltiplicazioni e somme, non come un `for`. Il secondo è il limite di complessità: un singolo programma troppo grande, con troppi percorsi possibili, viene rifiutato indipendentemente dalla sua correttezza, e l'unica soluzione è spezzarlo in più programmi più piccoli. C'è inoltre un limite di 512 byte di stack per ogni singola funzione, che diventerà rilevante nel capitolo dedicato all'evoluzione della pipeline hardcoded.

1.2 Punti di aggancio, mappe, XDP e tail call

eBPF può agganciarsi in molti punti diversi del kernel. Un *kprobe* intercetta l'ingresso o l'uscita da una funzione del kernel o da una syscall, ed è tipicamente usato per tracing e osservabilità. I *tracepoint* sono punti di strumentazione stabili predefiniti nel kernel. Ai fini di questo lavoro contano soprattutto due meccanismi: le mappe e l'hook XDP, più la primitiva di tail call.

Le *mappe* (*maps*) sono strutture dati che vivono nel kernel ma sono accessibili sia da altri programmi eBPF sia da user space. Sono il modo standard per mantenere stato tra un'esecuzione e l'altra di un programma – un programma eBPF non ha memoria persistente propria oltre alle mappe – e per far comunicare kernel e user space, che leggono e scrivono le stesse strutture. Esistono molti tipi di mappa: array indicizzati da interi, tabelle hash, array per-CPU (in cui ogni CPU ha la propria copia indipendente, evitando qualunque necessità di lock), e mappe speciali come `BPF_PROG_ARRAY`, che contiene riferimenti ad altri programmi eBPF. Tutte le mappe usate in questo lavoro – per i pesi del modello, per lo stato dei link, per le statistiche, per la tabella di forwarding – sono di questi tipi.

XDP (eXpress Data Path) [1] è il punto di aggancio più vicino alla scheda di rete: il programma eBPF viene eseguito nel driver stesso, prima ancora che il kernel costruisca le proprie strutture dati (l'`sk_buff`) per il pacchetto in arrivo. È quindi il punto più veloce in assoluto per ispezionare, scartare o reindirizzare un pacchetto, ed è esattamente per questo che è la scelta naturale per un data plane ad alte prestazioni. Un programma XDP riceve un puntatore al pacchetto grezzo e restituisce uno di quattro verdetti principali: `XDP_PASS` lascia proseguire il pacchetto verso lo stack di rete; `XDP_DROP` lo distrugge immediatamente; `XDP_TX` lo rispedisce sull'interfaccia da cui è arrivato; `XDP_REDIRECT` lo devia verso un'altra interfaccia. Quest'ultimo, tramite la helper `bpf_redirect()`, è il verdetto su cui si basa tutto il forwarding di questo progetto.

La *tail call* è il meccanismo che permette a un programma eBPF di cedere il controllo a un altro programma eBPF a runtime, tramite una `BPF_PROG_ARRAY` e la primitiva `bpf_tail_call()`. Se il salto riesce, l'esecuzione non torna mai indietro al chiamante: è più simile a una `exec` che a una chiamata di funzione. Questo meccanismo ha due usi complementari in questa tesi. Da un lato permette di costruire pipeline modulari, spezzando la logica in più programmi indipendenti; dall'altro è a volte l'unico modo di aggirare il limite di complessità del verifier, dividendo un blocco di codice troppo grande in pezzi che, presi singolarmente, passano la verifica.

Un ultimo dettaglio che ricorre in tutto il codice è quella che nelle presentazioni preparate durante il progetto abbiamo chiamato la “regola d'oro” del parsing: ogni volta che si accede a un campo di un header, bisogna prima verificare esplicitamente che il puntatore non superi `data_end`, il puntatore alla fine del buffer del pacchetto. Il verifier non ha alcuna garanzia a priori su quanti byte siano effettivamente disponibili, e senza questo controllo rifiuta il programma.

Listing 1.1. La regola d'oro: controllo dei limiti prima di ogni accesso

```
1 void *data      = (void *) (long) ctx->data;
2 void *data_end  = (void *) (long) ctx->data_end;
3 struct ethhdr *eth = data;
4 if ((void *) (eth + 1) > data_end)
5     return XDP_PASS;
6 if (eth->h_proto == bpf_htons(ETH_P_IP)) {
7     struct iphdr *ip = (void *) (eth + 1);
8     if ((void *) (ip + 1) > data_end)
9         return XDP_PASS;
10 }
```

Su questi mattoni – mappe, XDP, tail call, parsing sicuro – è costruito tutto il resto: un protocollo applicativo personalizzato incapsulato dentro UDP (un header “IPA” che viaggia dopo l'header UDP, su una porta convenzionale) e, sopra di esso, un intero sistema di inferenza neurale eseguito interamente nel kernel.

1.3 Il paradigma Intelligent Packets

Il progetto IPA nasce da un'idea radicale rispetto al modo classico di pensare a un router programmabile. Nel modello tradizionale un control plane centrale calcola le regole di forwarding e le installa nei nodi, che si limitano a eseguirle; nel paradigma Intelligent Packets, invece, è il pacchetto stesso a portare con sé l'informazione necessaria per essere instradato. Un modello di machine learning compatto – nel nostro caso un percettrore multistrato (MLP) con pesi quantizzati a 8 bit – viene incorporato nell'header del pacchetto oppure registrato nel nodo, e ad ogni salto il router esegue una mini-inferenza locale per decidere su quale interfaccia inoltrare il pacchetto, in totale autonomia. Missioni di rete diverse – recupero da un guasto, rispetto di una deadline, gestione della congestione – diventano semplicemente modelli diversi, senza che l'operatore debba riconfigurare regole o tunnel.

Questo lavoro parte da una dimostrazione preliminare del paradigma, sviluppata precedentemente, che serve da premessa concettuale e vale la pena riassumere brevemente. In quella prima fase l'inferenza nel kernel era ridotta al minimo – un

prodotto scalare a quattro termini tra un piccolo input vector (`model_id`, TTL del pacchetto, interfaccia di ingresso e dimensione dell'input) e quattro pesi – e il cuore dell'esperimento era mostrare che si potesse chiudere il ciclo completo interamente nel data plane. Lo switch era organizzato attorno a due mappe: `model_cache`, che associa un `model_id` ai pesi del modello, e `fwd_table`, che associa il risultato dell'inferenza a un'azione di forwarding concreta (interfaccia di uscita e MAC da riscrivere). Su questa base erano stati confrontati quattro metodi di popolamento delle tabelle, ciascuno dei quali metteva in luce un aspetto diverso del problema.

Il primo metodo, basato su quantizzazione post-training (PTQ), addestrava il modello in virgola mobile e lo quantizzava solo a posteriori, con il control plane che usava i pesi float originali e il kernel quelli interi: un disallineamento intenzionale, che produceva errori di forwarding misurabili e serviva proprio a quantificare la perdita di accuratezza della quantizzazione. Il secondo, basato su addestramento consapevole della quantizzazione (QAT), addestrava il modello direttamente in interi con fake-quantization, ottenendo un'aritmetica identica tra control plane e kernel e quindi un allineamento perfetto: chiavi sempre corrette, nessun errore. Il terzo metodo popolava la tabella di forwarding in modo reattivo, in stile OpenFlow: la tabella parte vuota e ogni fallimento di lookup genera un evento verso il control plane, che installa la regola usando direttamente la chiave calcolata dal kernel, eliminando alla radice ogni rischio di disallineamento. Il quarto, la vera dimostrazione del paradigma, faceva viaggiare il modello *dentro* il pacchetto: sia la cache dei modelli sia la tabella di forwarding partono completamente vuote, il primo pacchetto per un dato modello porta i pesi nel proprio payload, il nodo li estrae al volo, popola la cache e installa la prima regola, e da quel momento in poi inoltra in autonomia a velocità kernel, senza più coinvolgere il control plane.

La lezione che quella fase preliminare ha lasciato, e che ha guidato tutto il lavoro descritto in questa tesi, è duplice. Da un lato ha dimostrato che l'idea funziona end-to-end: l'intelligenza può stare nel pacchetto, il nodo può apprendere al volo e forwardare autonomamente. Dall'altro ha usato un modello volutamente minimale – un singolo prodotto scalare a quattro termini – che non è un vero modello neurale multistrato e non solleva nessuna delle domande interessanti su *come* eseguire efficientemente una rete neurale vera dentro il datapath. È esattamente questo il punto di partenza del presente lavoro: generalizzare l'inferenza a un MLP multistrato quantizzato, ed esplorare sistematicamente i modi diversi di eseguirlo nel kernel, con il loro preciso compromesso tra prestazioni e flessibilità. Vale la pena segnalare che la direzione di eseguire inferenza neurale direttamente nel kernel non è isolata a questo progetto: esistono lavori recenti e indipendenti che esplorano la stessa idea, dall'inferenza di reti convoluzionali leggere in eBPF per la classificazione del traffico su dispositivi edge [4], ai classificatori basati su reti neurali quantizzate e alberi decisionali eseguiti sia in kernel sia in user space per il packet processing [5].

1.4 Ambiente sperimentale

Tutti gli esperimenti sono stati condotti su *Kathara*, uno strumento di emulazione di rete che fa corrispondere ogni nodo di una topologia a un container Docker leggero. La differenza rispetto a una macchina virtuale completa non è di poco conto: un

container condivide il kernel del sistema host, ed è proprio questo che rende possibile eseguire programmi eBPF/XDP dentro ciascun nodo emulato. Ogni nodo, all'avvio, monta il filesystem `debugfs`, abilita l'inoltro IP e avvia il demone di routing FRR con protocollo OSPF.

La topologia usata è *Germany50*, importata dal repository SNDlib, una libreria di topologie reali comunemente usata in letteratura: una dorsale di cinquanta nodi, a cui si aggiungono due host virtuali di sorgente e destinazione, con grado massimo di un singolo nodo pari a sei – un numero non casuale, perché è esattamente il numero di interfacce di uscita che il modello neurale deve poter scegliere, e quindi il numero di classi in uscita dall'argmax finale (sei interfacce più la classe di scarto). Il traffico di prova viaggia su UDP verso la porta 9999, dove è in ascolto il protocollo IPA.

Un limite pratico di questo fabric, incontrato durante la validazione, condiziona diverse scelte descritte più avanti: il traffico UDP sulla porta 9999 non viene consegnato end-to-end in ogni condizione. In particolare, un pacchetto indirizzato all'indirizzo di loopback di un nodo distante più di un salto semplicemente non arriva, mentre uno indirizzato all'indirizzo IP dell'interfaccia direttamente connessa di un vicino sì. Per questo motivo la verifica sistematica di correttezza non si appoggia a traffico di rete reale, ma alla primitiva `BPF_PROG_TEST_RUN`, che permette di eseguire un programma XDP già caricato su un pacchetto sintetico direttamente dal kernel, ottenendo sia il verdetto del programma sia una misura di durata, senza attraversare nessuna scheda di rete.

Capitolo 2

Il design space: tre pipeline per l’inferenza neurale nel datapath

Il cuore di questa tesi è un *design space* a tre punti: tre modi diversi di eseguire lo stesso modello neurale – un MLP con 65 input, due hidden layer da 4 neuroni ciascuno e 7 output – dentro il datapath eBPF/XDP. I tre punti si chiamano, nel codice e in questa tesi, P1 hardcoded, P2 template e P3 modular, e si differenziano per quanto il codice generato sia specializzato sul singolo modello rispetto a quanto sia generico e parametrizzabile a runtime. Come si vedrà, più si specializza il codice più aumenta la velocità di picco ma diminuisce la flessibilità di aggiornamento; viceversa, aumentando la flessibilità crescono il numero di tail call, di letture da mappa e di istruzioni eBPF eseguite per pacchetto, e il throughput massimo cala di conseguenza. Non esiste, cioè, un’implementazione “naturale” unica: esiste una scelta esplicita su quanto specializzare il datapath, e questo capitolo la esplora in tutti e tre i suoi estremi, misurandone il costo.

2.1 Architettura comune alle tre pipeline

Le tre pipeline condividono lo stesso protocollo applicativo e la stessa famiglia di modelli. Il pacchetto IPA viaggia dentro UDP sulla porta 9999 e porta un header custom con `model_id`, `scale_factor` e un descrittore delle feature usate, su cui si tornerà diffusamente nel Capitolo 3. Il programma XDP intercetta il pacchetto in ingresso, prima ancora che raggiunga lo stack di rete Linux, ed esegue un perceptrone multistrato (MLP) quantizzato post-training a interi a 8 bit, con *aritmetica interamente intera* nel kernel – nessun numero in virgola mobile compare mai nel codice eseguito in kernel space, sia perché il verifier non gestisce operazioni floating point, sia perché l’aritmetica intera è deterministica e riproducibile. L’output finale è un argmax su un numero di classi pari al numero di interfacce di uscita più uno: nel nostro scenario, sei interfacce fisiche più la classe di scarto (DROP), per un totale di sette.

L’architettura del modello *non* è fissa: come si vedrà, la pipeline template varia a runtime le larghezze degli strati, la pipeline modular varia a runtime sia la larghezza sia la profondità, e la pipeline hardcoded – dopo il lavoro descritto nel Capitolo 3 – supporta un numero arbitrario di strati. Ai fini del confronto sperimentale tra le tre

pipeline, però, serve un modello *comune* da eseguire nelle stesse condizioni su tutte e tre, altrimenti le differenze misurate confonderebbero l'effetto della pipeline con l'effetto di architetture diverse. Come modello di riferimento per tutte le misure comparative di questo capitolo si usa quindi lo stesso MLP $65 \rightarrow 4 \rightarrow 4 \rightarrow 7$ – sessantacinque feature in ingresso, due hidden layer da quattro neuroni, sette classi in uscita – scelto perché è l'architettura effettivamente addestrata per lo scenario di routing studiato. È un riferimento di misura, non un vincolo dell'implementazione.

L'input vector di 65 componenti è costruito da quattro tipi di feature concatenate, ciascuna con un significato preciso legato allo scenario di routing. Le prime 6 componenti (**link_state**) rappresentano lo stato up/down delle sei interfacce di uscita del nodo; le successive 6 (**iface**, codificata one-hot) indicano da quale porta è arrivato il pacchetto; la tredicesima componente è il TTL del pacchetto, letto come scalare; le ultime 52 (**node**, anch'essa one-hot) identificano il nodo corrente nella topologia. Il vettore è quindi molto sparso: a runtime solo una manciata di posizioni è effettivamente diversa da zero, una proprietà che, come si vedrà, le tre pipeline sfruttano tutte – costruiscono il vettore feature per feature invece di materializzare un array denso di 65 elementi, che per giunta sfiorerebbe da solo buona parte del budget di stack disponibile.

Un aspetto su cui si è lavorato esplicitamente è rendere *vera* la prima feature, **link_state**. In una prima versione questa feature era sempre a 1 (tutti i link sempre su), il che significa che il modello non vedeva mai un guasto reale e quindi non poteva mai imparare a reagire ad esso. È stato introdotto un thread in user space, **link_state_monitor.py**, che interroga periodicamente lo stato del carrier di ogni interfaccia (leggendo `/sys/class/net/ethX/carrier`) e scrive il valore corrispondente nella mappa BPF condivisa **link_state[0..5]**. I programmi eBPF leggono questo valore ad ogni pacchetto, senza mai dover essere ricaricati: se un'interfaccia cade, la sua feature passa a 0 al polling successivo (che avviene ogni mezzo secondo), e questo può far sì che l'argmax scelga un'uscita diversa – è il meccanismo che nella documentazione del progetto chiamiamo *fast-reroute*. All'avvio la mappa viene inizializzata con tutti i link su, come stato di base sano.

Lo schema di elaborazione comune a tutte e tre le pipeline è riassunto nella Figura 2.1: cambia soltanto il blocco centrale, cioè *come* l'inferenza viene eseguita, non il parsing a monte né l'azione a valle.

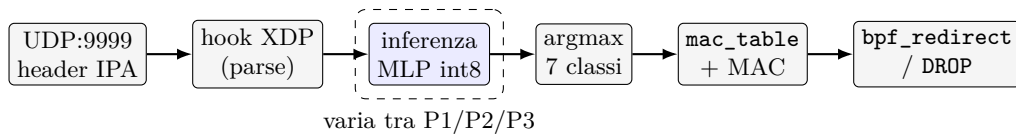


Figura 2.1. Il flusso del pacchetto comune alle tre pipeline. Il solo blocco di inferenza (tratteggiato) cambia implementazione; parsing e azione di forwarding sono identici.

2.2 Pipeline 1: hardcoded

Nella pipeline hardcoded i pesi della rete neurale sono incorporati come *letterali* direttamente nel sorgente C che viene generato e compilato per quel modello specifico.

Non esiste nessuna mappa dei pesi: ogni peso è una costante scritta nel codice, e l'intera inferenza è una lunga sequenza di operazioni aritmetiche su costanti note. Un dispatcher molto leggero legge il `model_id` dal pacchetto ed esegue un'unica tail call verso il programma `model_<id>` corrispondente; da lì in poi la costruzione dell'input vector, i due hidden layer, l'output layer e l'argmax sono completamente srotolati, senza un solo ciclo e senza una sola lettura da mappa per i pesi. Le uniche letture da mappa che sopravvivono sono quelle di `link_state` – che è una feature di input, non un peso – e, a valle dell'argmax, quella su `mac_table` per risolvere l'azione di forwarding.

Il fatto che i pesi siano costanti note al compilatore ha un beneficio prestazionale non ovvio: il compilatore applica automaticamente la *strength reduction*. Una moltiplicazione per un peso pari a zero scompare del tutto dal codice generato, una moltiplicazione per una potenza di due diventa uno shift, e in generale molte operazioni aritmetiche si semplificano o si eliminano già a compile time. Dato che i pesi quantizzati contengono tipicamente molti zeri e molti valori piccoli, questa ottimizzazione riduce sensibilmente il numero di istruzioni effettivamente eseguite per pacchetto – un vantaggio che nessuna delle altre due pipeline, che leggono i pesi da una mappa a runtime, può sfruttare.

Il dettaglio tecnicamente più delicato riguarda le due feature codificate one-hot, `iface` e `node`. L'idea più naturale per usarle – un array di pesi statico indicizzato dalla posizione attiva, calcolata a runtime – non è accettata dal verifier: un `static const` indicizzato da una variabile non risolvibile staticamente genera, in fase di caricamento, una relocation che il loader non sa risolvere e che collassa silenziosamente a zero, con il risultato che il modello si comporterebbe come se quella feature non contasse mai nulla, senza che il verifier segnali alcunché di anomalo. La soluzione adottata è un unico blocco `switch` per ciascuna feature one-hot – uno per `iface`, uno per `node` – che, in base alla posizione attiva, assegna in un colpo solo il contributo di quella feature a *tutti* i neuroni del primo hidden layer. La soluzione alternativa, uno switch per ogni singolo neurone, è più naturale da scrivere ma genera un'esplosione combinatoria di rami – dell'ordine di $(7 \cdot 52)^{\text{neuroni}}$ percorsi possibili – che il verifier arriva a rifiutare per limite di complessità. Un singolo blocco che seleziona il contributo di tutti i neuroni insieme riduce invece il costo a $O(7 + 52)$ rami, e passa la verifica.

Il Listato 2.1 mostra come tutto questo si concretizza nel codice generato per un singolo neurone del primo hidden layer. Le componenti `link_state` sono lette da una mappa (feature di input); i contributi delle due feature one-hot arrivano dalle variabili `w_iface_j` e `w_node_j` preselezionate dai rispettivi switch; il TTL entra come scalare; tutti i pesi restanti sono letterali cablati nel codice, e la funzione `RELU_LL` applica l'attivazione mantenendo l'aritmetica interamente intera a 64 bit per evitare overflow durante l'accumulo.

Listing 2.1. Un neurone del primo hidden layer, generato per la pipeline hardcoded

```

1  /* link_state[0..5] letti dalla mappa (feature, non pesi) */
2  long long h1_0 = RELU_LL(
3      (__s64)_ttl * 12LL          /* TTL, scalare, peso
                                   letterale */

```

```

4      + w_iface_0 + w_node_0          /* one-hot: contributi
      preselezionati */
5      + ls0*-7LL + ls1*3LL + ls2*0LL    /* link_state * pesi
      letterali */
6      + ls3*5LL + ls4*-2LL + ls5*1LL
7      + 4LL );                        /* bias, letterale */

```

Si noti il termine `ls2*0LL`: è un peso nullo, che il compilatore elimina del tutto grazie alla strength reduction, come farà anche con le moltiplicazioni per potenze di due. Nessuna delle altre due pipeline, che leggono i pesi da una mappa a runtime, può beneficiare di questa semplificazione a compile time, perché per il compilatore quei pesi sono valori sconosciuti fino all'esecuzione.

Il costo di aggiornare un modello in questa pipeline è, semplicemente, il costo di rigenerare il sorgente C, ricompilarlo e ricaricare l'intero programma XDP: non esiste alcuna via incrementale, per costruzione, dato che i pesi sono letterali nel codice sorgente. È la specializzazione massima e – come confermeranno i risultati sperimentali – anche le prestazioni massime. Proprio questo costo di aggiornamento, però, è il limite principale su cui interverrà il Capitolo 3.

2.3 Pipeline 2: template

La pipeline template sposta i pesi fuori dal codice sorgente e dentro una mappa BPF (`arch_weights`, un `BPF_ARRAY`), lasciando invece fisso il *codice* che implementa un'intera famiglia di architetture. Tutti i modelli con la stessa struttura generale – due hidden layer, input e output di dimensione fissata dal protocollo – condividono lo stesso programma compilato, `arch_generic_2layer`, che viene caricato una sola volta. Le larghezze dei due hidden layer, indicate nel codice come `n_h1` e `n_h2`, non sono più costanti di compilazione ma vengono lette a runtime da una mappa di registro (`arch_registry`), fino a un tetto massimo fissato in fase di compilazione: cambiare la larghezza di un modello, o registrarne uno completamente diverso ma con la stessa profondità, non richiede alcuna ricompilazione, solo una scrittura nelle mappe dei pesi e del registro.

Il prodotto scalare del primo hidden layer, invece di sommare termini letterali come in P1, itera su un ciclo srotolato che ad ogni iterazione legge un peso dalla mappa `arch_weights` a un offset calcolato dinamicamente a partire da `n_h1` e `n_h2`. Il punto sottile è *perché* il verifier accetti un indice calcolato a runtime qui, quando lo rifiutava per l'array statico di P1: la differenza è che `arch_weights` è una mappa BPF regolarmente registrata, cioè un oggetto kernel con dimensioni note e controllate dal verifier, non una relocation risolta a compile time. L'aritmetica dell'indice è sicura perché ogni accesso alla mappa è comunque validato dal kernel contro i limiti dichiarati della mappa stessa; non c'è nessun puntatore non risolto e nessuna esplosione di rami. È esattamente questo spostamento – dai letterali nel codice a una mappa registrata – che compra la flessibilità di P2 al prezzo di una lettura di mappa per ogni peso.

Per il primo layer, che deve costruire l'input vector, la lettura di `link_state` avviene dentro lo stesso ciclo: ad ogni neurone j , il contributo di ogni componente i del `link_state` è `link_state[i]` moltiplicato per il peso corrispondente letto dalla

mappa. Aggiornare un modello, o cambiarne la larghezza, significa quindi soltanto sovrascrivere i pesi nella mappa e la voce di registro: nessuna ricompilazione, nessun reload del programma. Abbiamo verificato concretamente questa flessibilità registrando, nello stesso oggetto BPF compilato, due modelli con forma diversa – il modello reale 65-4-4-7 insieme a un modello sintetico 65-6-5-7, stessa profondità ma larghezza diversa – e ottenendo per entrambi la classe corretta, confrontata con un riferimento indipendente in Python. È la prova che la registrazione concorrente di architetture diverse nello stesso programma funziona davvero, e non solo in linea di principio.

2.4 Pipeline 3: modular

La pipeline modular porta il principio di P2 ancora oltre, rendendo dinamica non solo la larghezza ma anche la *profondità* del modello. L’inferenza è scomposta in una catena di tail call attraverso due soli programmi generici. Il primo, `layer_first`, è sempre eseguito al primo hop e fa una lettura sparsa del vettore di 65 feature, esattamente come in P1 e P2. Il secondo, `layer_hidden`, è eseguito per ogni hop successivo e calcola un prodotto denso $n_{in} \rightarrow n_{out}$ tra dimensioni lette a runtime da un registro (`layer_registry/layer_shapes`). Le attivazioni intermedie – l’output di un layer che serve da input al layer successivo – transitano attraverso una mappa di scratch per-CPU (`PERCPU_ARRAY`), in modo che CPU diverse non si contendano mai lo stesso stato e non serva alcun meccanismo di locking. Il dispatcher, prima di avviare la catena, scrive solo pochi valori di metadato (il `model_id`, lo scale factor, l’indice di layer iniziale, l’interfaccia di ingresso e il TTL), non l’intero vettore denso.

Un punto concettualmente importante è che quale sia l’*ultimo* layer non è deciso da quale slot della tail-call array è occupato, ma dai dati stessi: ogni hop controlla se il proprio indice di layer, più uno, coincide con il numero totale di layer del modello corrente, letto dal registro. Se coincide, calcola l’argmax e forwarda; altrimenti scrive la propria attivazione nello scratch e fa tail call verso l’hop successivo. Questo permette a modelli con profondità diversa di condividere esattamente la stessa catena di tail call – lo slot 0 è sempre `layer_first`, gli slot successivi sempre `layer_hidden` – senza nessun conflitto e senza dover ricablare nulla per cambiare profondità. Anche qui il caso limite è stato validato concretamente: due modelli registrati nello stesso oggetto compilato, quello reale a 3 layer (65-4-4-7) e uno sintetico a 4 layer (65-5-6-4-7, con sia profondità sia larghezza diverse), entrambi corretti.

È proprio in questa pipeline che si è incontrato in modo concreto il limite di complessità del verifier discusso nel primo capitolo: un primo tentativo di scrivere un unico blocco denso per l’intera inferenza compilava a circa 19983 istruzioni, contro un limite di 4096 imposto dal kernel di laboratorio. La separazione tra `layer_first` (lettura sparsa iniziale, relativamente leggera) e `layer_hidden` (prodotto denso, ripetuto quante volte serve via tail call) non è quindi solo un’eleganza architeturale, ma la condizione necessaria per tenere ciascun pezzo sotto il limite. Il prezzo di questa flessibilità massima è un numero di tail call per pacchetto che dipende dalla profondità del modello – tre, per il modello di riferimento a tre layer – contro l’unica

tail call di P1 e P2, e questo, come si vedrà, è il fattore che pesa di più sulle sue prestazioni.

2.5 Dall'argmax all'azione: una mappa comune

Una volta calcolato l'argmax, le tre pipeline condividono lo stesso schema per tradurlo in un'azione di rete concreta. La classe scelta – da 0 a 5 per le interfacce, 6 per DROP – viene usata come chiave in una mappa `mac_table`, che restituisce l'indice dell'interfaccia di uscita insieme ai MAC address sorgente e destinazione da scrivere nell'header Ethernet prima del redirect. In P1 questa risoluzione avviene tramite uno switch cablato, senza letture di mappa aggiuntive; in P2 e P3 tramite un'unica lettura su `mac_table`. Non viene fatta nessuna validazione ulteriore sull'output del modello: ci si fida della decisione della rete neurale, coerentemente con lo spirito del paradigma IPA descritto nel primo capitolo. La Figura 2.2 mette a confronto, schematicamente, i tre modi di eseguire la stessa inferenza: dove stanno i pesi, quante tail call servono e cosa si aggiorna per cambiare modello.

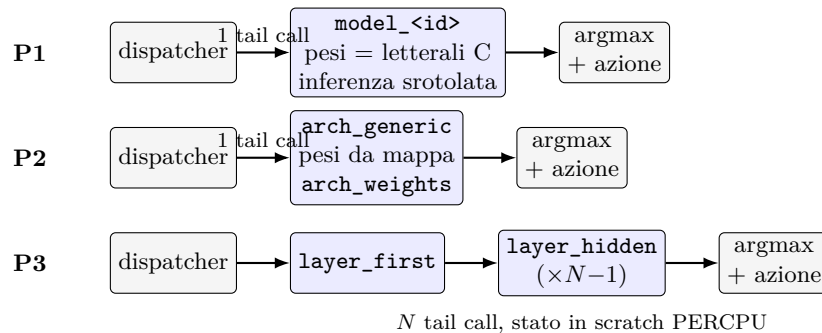


Figura 2.2. Le tre pipeline a confronto. P1 incorpora i pesi come letterali (nessuna lettura di pesi, aggiornamento = ricompila); P2 tiene i pesi in una mappa (aggiornamento = scrittura mappa); P3 scompone la rete in una catena di tail call generiche (profondità e larghezza dinamiche).

2.6 Le tre pipeline a confronto

Prima di passare ai numeri vale la pena fermarsi sul senso complessivo delle tre pipeline, perché il modo migliore per capirle non è tanto guardarle una per una quanto vederle come una *progressione*: ciascuna rilassa un vincolo in più della precedente, e ogni vincolo rilassato ha un prezzo prestazionale preciso. È questa progressione a rendere il design space uno spazio di scelta ben ordinato e non una collezione di tre implementazioni scorrelate.

Il punto di partenza è **che cosa è fisso e che cosa è variabile senza dover ricompilare**. Nella pipeline hardcoded è fisso tutto: i pesi sono costanti nel codice, e per cambiare anche un solo peso – non parliamo dell'architettura – bisogna rigenerare il sorgente, ricompilarlo e ricaricarlo. Questa rigidità è però esattamente ciò che le dà il suo vantaggio: siccome il compilatore conosce ogni peso, può semplificare via l'aritmetica inutile (la strength reduction discussa sopra), un'ottimizzazione che

nessuna delle altre due può sfruttare perché in entrambe i pesi sono valori che il compilatore non vede, ignoti fino al momento dell'esecuzione. La pipeline *hardcoded*, in altre parole, “paga” la propria specializzazione una volta sola, a tempo di compilazione, e ne raccoglie il beneficio ad ogni pacchetto.

La pipeline *template* compie il primo passo di flessibilizzazione: separa i pesi dal codice, spostandoli in una mappa. Questo cambia radicalmente il costo di aggiornamento – da secondi (ricompilazione) a millisecondi (una scrittura in mappa) – e permette di cambiare i pesi, o la larghezza degli hidden layer, di un modello mentre il datapath è vivo, senza mai ricaricare il programma. Ciò che la *template* *non* può fare, però, è cambiare la *profondità*: il suo unico programma è scritto per l'architettura “due hidden layer”, e un modello con tre o quattro strati richiederebbe un programma diverso. È il compromesso intermedio: molta più flessibilità dell'*hardcoded*, a un costo per pacchetto sensibilmente più alto (una lettura di mappa per ciascun peso, invece di zero), ma con un vincolo architetturale ancora presente.

La pipeline *modular* rimuove anche quest'ultimo vincolo. Scomponendo la rete in una catena di layer generici concatenati via tail call, e leggendo da un registro quanti layer ha ciascun modello, rende dinamica anche la profondità: lo stesso identico oggetto compilato serve modelli a due, tre o quattro strati senza alcuna modifica. È la massima flessibilità concepibile in questo schema – si può cambiare qualunque aspetto dell'architettura scrivendo solo delle mappe – ma è anche quella che paga il prezzo più alto per pacchetto, perché ogni strato in più è una tail call in più (con il suo doppio parsing e il suo passaggio attraverso lo scratch per-CPU delle attivazioni intermedie), e il numero di letture di mappa cresce di conseguenza.

La Tabella 2.1 riassume questa progressione lungo gli assi che contano davvero. Si legga da sinistra a destra come un aumento monotono di flessibilità, e dall'alto verso il basso come i diversi “prezzi” che quella flessibilità comporta.

Tabella 2.1. Le tre pipeline lungo gli assi che le distinguono. Da P1 a P3 la flessibilità cresce, e con essa il costo per pacchetto; specularmente cala il costo di aggiornamento del modello.

	P1 <i>hardcoded</i>	P2 <i>template</i>	P3 <i>modular</i>
Dove stanno i pesi	letterali nel codice C	mappa <code>arch_weights</code>	mappa <code>layer_weights</code>
Variabile senza ricompilare	nulla	larghezza degli strati	larghezza e profondità
Programmi eBPF compilati	uno per modello	uno per l'intera famiglia	due, generici
Tail call per pacchetto	1	1	N (= profondità)
Stato intermedio	nessuno (tutto srotolato)	nessuno	scratch per-CPU
Strength reduction	sì (pesi noti)	no	no
Aggiornamento modello	ricompila + reload (~1.1 s)	scrittura mappa (~1.6 ms)	scrittura mappa (~1.1 ms)
Flessibilità	bassa	media	alta
Prestazioni per pacchetto	massime	intermedie	minime

Vale la pena sottolineare un'asimmetria che questa tabella rende evidente: le due metriche di costo – costo per pacchetto e costo di aggiornamento del modello – si muovono in direzioni opposte lungo la progressione. La pipeline *hardcoded* è

imbattibile sul primo asse e pessima sul secondo; la modular è l'esatto contrario; la template sta in mezzo su entrambi. Non esiste una pipeline che vinca su entrambi gli assi contemporaneamente, ed è precisamente questa tensione – non un difetto di implementazione, ma una proprietà strutturale del problema – a definire il design space e a rendere la scelta della pipeline una decisione che dipende dallo scenario d'uso, non una questione di qualità del codice. Lo scenario studiato in questa tesi – modelli noti in anticipo, che cambiano di rado – ricade nettamente dalla parte in cui il costo per pacchetto conta più di quello di aggiornamento, ed è la ragione per cui, come si argomenterà dopo aver visto i numeri, la pipeline hardcoded è la scelta vincente.

2.7 Risultati sperimentali comparativi

Tutte e tre le pipeline sono state misurate con la stessa metodologia, basata su `BPF_PROG_TEST_RUN` per le ragioni discusse nella Sezione 1.4. Accanto alle tre è stata misurata anche una quarta configurazione, chiamata *baseline*: un programma che fa lo stesso parsing del dispatcher e lo stesso redirect finale, ma senza nessuna tail call e senza nessuna inferenza. Serve da pavimento di riferimento, per capire quanto della latenza osservata sia dovuto davvero al modello neurale e quanto al solo framework XDP. Il criterio di correttezza usato in tutta la suite è robusto e semplice da enunciare: si pre-installa la mappa di forwarding con le azioni attese, si esegue il programma e si controlla che il verdetto sia coerente con un redirect o un drop e che il contatore per-classe della classe scelta si sia effettivamente incrementato, confrontando sempre la classe scelta dal kernel con quella calcolata indipendentemente da un riferimento in Python che replica esattamente la stessa aritmetica intera.

I numeri riportati nella Tabella 2.2 meritano una nota metodologica importante, perché il modo in cui sono stati ottenuti è parte del contributo. Il primo tentativo di misurare la latenza con un singolo campione di `BPF_PROG_TEST_RUN` ha prodotto risultati estremamente instabili: la stessa identica configurazione, misurata due volte a distanza di pochi minuti, poteva restituire valori che differivano anche di un fattore venti, senza alcuna correlazione con il numero di istruzioni. La spiegazione è che il rumore introdotto da un ambiente condiviso – interruzioni dello scheduler, altri processi sulla stessa macchina, variazioni di frequenza della CPU – è *a senso unico*: può solo rallentare un campione, mai accelerarlo al di sotto del suo vero costo. Questo rende il *minimo* su più campioni indipendente la statistica corretta da riportare, non la media né la mediana – lo stesso principio su cui si basano strumenti di benchmarking consolidati come *hyperfine* o Google Benchmark. Il problema si inserisce, del resto, in una casistica ben nota di errori metodologici che invalidano i confronti sperimentali in sistemi di questo tipo, catalogata sistematicamente da Heiser [2] e descritta concretamente per il caso specifico di XDP in un intervento a USENIX LISA 2021 [3], dove viene mostrato un caso in cui l'utilizzo di CPU riportato dagli strumenti standard sottostimava il costo reale di un fattore sessanta. Va anche detto, con onestà, che l'ambiente di laboratorio usato – una macchina virtuale, senza controllo su frequenza della CPU, C-state o affinità delle interruzioni – rende i numeri assoluti qui riportati non direttamente comparabili con quelli di un paper su hardware dedicato; ciò che resta pienamente difendibile, ed è la base

di tutte le conclusioni di questa tesi, è il confronto *relativo* tra le pipeline misurate sullo stesso nodo nelle stesse condizioni. Dopo aver corretto la suite facendo eseguire ciascuna misura sette volte e riportando il minimo, i numeri sono risultati stabili tra esecuzioni successive.

Tabella 2.2. Confronto sperimentale tra le quattro configurazioni (kernel reale, BPF_PROG_TEST_RUN, minimo su 7 trial indipendenti).

Metrica	baseline	P1 hardcoded	P2 template	P3 modular
Istruzioni eBPF (xlated)	113	980	16 988	15 767
Codice jited (byte)	542	4 684	84 587	76 376
Tail call / pacchetto	0	1	1	3
Lettura mappa / pacchetto (reali)	0	4.0	147.0	160.0
Memoria mappe (byte)	280	308	8 052	16 884
Latenza minima (ns/pacchetto)	21.0	35.0	262.0	442.0
Throughput (Mpps, da latenza minima)	47.619	28.571	3.817	2.262
CPU (%)	34	30	57	51

La lettura è coerente con la tassonomia proposta: il costo – in istruzioni, codice jited, tail call, letture di mappa e memoria – cresce in modo monotono passando da baseline a P1 a P2 a P3, e le prestazioni calano nello stesso ordine. Il baseline, che rappresenta il solo pavimento di parsing e redirect senza nessuna inferenza, misura 21 nanosecondi per pacchetto e 47.6 milioni di pacchetti al secondo; la pipeline hardcoded aggiunge appena circa 14 nanosecondi – per la tail call, il secondo parsing che essa comporta e l'intera rete neurale – arrivando a 35 nanosecondi, cioè circa 1.7 volte il pavimento assoluto, non ordini di grandezza. Questo numero risponde direttamente a una domanda posta dal relatore, cioè se il throughput molto alto della pipeline hardcoded fosse in qualche modo sospetto: il confronto con il baseline mostra che quel throughput non è altro che il pavimento imposto dal framework XDP stesso, e che una rete int8 65-4-4-7 completamente srotolata costa pochissimo in confronto.

Un'osservazione interessante riguarda P2 e P3: pur avendo un conteggio statico di istruzioni molto simile, le loro prestazioni differiscono in modo netto, e la differenza si spiega quasi interamente con il numero di tail call (tre contro una) e con le letture di mappa aggiuntive, non con il volume di codice in sé – una conferma diretta che, in questo regime, sono le tail call e gli accessi a mappa, non le istruzioni aritmetiche, a dominare il costo per pacchetto.

Oltre alla prestazione per pacchetto è stato misurato, con chiamate reali a BPF_PROG_LOAD e `bpf_map_update_elem`, il costo di registrare un nuovo modello a runtime in ciascuna pipeline – la metrica duale, che racconta il lato “flessibilità” del compromesso. Per la pipeline hardcoded, che non ha alcuna via incrementale, aggiungere un modello significa rigenerare il sorgente C e ricompilarlo da zero: un'operazione che costa, in media, circa 1.1 secondi. Per template e modular, che si limitano a scrivere le nuove mappe di pesi, lo stesso aggiornamento costa rispettivamente circa 1.6 e 1.1 millisecondi: da 700 a mille volte più economico. È esattamente il prezzo della flessibilità – template e modular pagano un costo per pacchetto più alto in cambio di un aggiornamento del modello quasi gratuito – ed è il compromesso che tiene insieme tutto il design space.

Per completezza va detto che la suite verifica correttezza e prestazioni non solo sul modello di riferimento 65-4-4-7, ma anche su architetture di forma diversa, proprio per non lasciare scoperta l'affermazione più forte fatta in questo capitolo. Per la pipeline hardcoded vengono compilati e verificati programmi con profondità diverse (uno con un solo hidden layer, uno con tre); per template e modular, un modello sintetico di forma diversa viene registrato insieme a quello reale nello stesso oggetto compilato. Tutte queste varianti passano il criterio di correttezza contro il riferimento Python.

2.8 Colli di bottiglia trasversali del datapath

Le differenze di prestazione tra le tre pipeline, discusse finora, dipendono da *quanto* lavoro ciascuna fa per pacchetto – numero di tail call, di letture di mappa, di istruzioni. Esiste però una seconda categoria di costi, indipendente dalla scelta della pipeline, che è comune a tutte e tre e vale la pena analizzare a un livello più basso, perché è proprio lì che si annidano le ottimizzazioni più promettenti e più facilmente trasferibili. Tutti i punti seguenti sono stati riscontrati identici nel codice C generato per P1, P2 e P3.

Il primo è il **doppio parsing** del pacchetto. Il dispatcher, prima di scegliere il programma da invocare, esegue il parsing completo degli header – Ethernet, IP, UDP e infine l'header IPA – solo per poter leggere un singolo byte, il `model_id`, e decidere verso quale `model_id` fare tail call:

Listing 2.2. Il dispatcher parse tutto solo per leggere `model_id`

```

1  /* ... controlli su eth, ip, ip_proto, ihl, udp->dest == 9999
   ... */
2  struct ipa_hdr *ipa = (struct ipa_hdr *) (udp + 1);
3  if ((void *) (ipa + 1) > data_end) return XDP_PASS;
4  __u32 mid = (__u32) ipa->model_id;
5  model_progs.call(ctx, mid);    /* tail call: i puntatori qui
   muoiono */

```

Il problema è che, dopo la tail call, il programma `model_id` deve rifare da zero esattamente lo stesso parsing. Questo non è una svista: i puntatori `data`, `data_end` e tutti quelli derivati non sopravvivono a una tail call – il programma chiamato riparte con il solo contesto `ctx` e deve riderivare `data/data_end` dal contesto e rieseguire tutti i controlli di limite, altrimenti il verifier rifiuta ogni accesso al pacchetto. Il risultato è che ogni pacchetto attraversa il parsing degli header due volte, ed è una componente concreta dei ~14 nanosecondi che separano la pipeline hardcoded dal baseline. Per il caso a modello singolo si potrebbe eliminare il dispatcher e attaccare direttamente il programma del modello, risparmiando sia la tail call sia il secondo parsing, al costo di rinunciare al dispatch multi-modello – un compromesso ragionevole proprio nel caso d'uso della pipeline hardcoded, dove i modelli sono noti in anticipo.

Il secondo è la scelta del tipo di mappa per `mac_table`, che traduce la classe scelta dall'argmax nell'azione di forwarding. In tutte e tre le pipeline questa mappa è dichiarata come `BPF_HASH`, con chiave un intero che rappresenta la classe (da 0 a 6):

Listing 2.3. `mac_table` come hash, con chiavi intere dense

```

1 BPF_HASH(mac_table, __u32, struct fwd_action, 8);
2 /* ... */
3 __u32 _cls = (__u32)best_cls; /* 0..6 */
4 struct fwd_action *_action = mac_table.lookup(&_cls);

```

Una tabella hash calcola una funzione di hash sulla chiave e poi risolve eventuali collisioni percorrendo un bucket – un costo giustificato quando le chiavi sono sparse o non note. Qui però le chiavi sono esattamente gli interi densi da 0 a 6: una `BPF_ARRAY`, indicizzata direttamente dalla classe, farebbe lo stesso lavoro con un semplice accesso a offset costante, senza calcolo di hash né gestione di collisioni. È un’ottimizzazione a costo di implementazione quasi nullo che si applica identica a tutte e tre le pipeline.

Il terzo, e probabilmente il più significativo su hardware reale multi-core, riguarda i contatori di statistica. Le mappe `pkt_stats` e `cls_stats` sono `BPF_ARRAY` condivise tra tutte le CPU, e vengono aggiornate ad ogni pacchetto con un’operazione atomica:

Listing 2.4. Contatori condivisi aggiornati con atomica per pacchetto

```

1 int _hi = 0; __u64 *_hv = pkt_stats.lookup(&_hi);
2 if (_hv) __sync_fetch_and_add(_hv, 1); /* atomica su cache-
   line condivisa */
3 __u64 *_cv = cls_stats.lookup(&_cls);
4 if (_cv) __sync_fetch_and_add(_cv, 1);

```

In un contesto XDP reale il traffico viene distribuito su più CPU tramite RSS (Receive Side Scaling), e ogni CPU che processa un pacchetto esegue queste atomiche sulla *stessa* riga di cache: il risultato è un continuo rimbalzo della cache-line tra i core (*cache-line bouncing*), un costo che cresce con il numero di CPU e che le misure su singola CPU con `BPF_PROG_TEST_RUN` non catturano affatto. La soluzione idiomatica in eBPF è usare `BPF_PERCPU_ARRAY`: ogni CPU aggiorna la propria copia privata del contatore senza alcuna atomica e senza contesa, e lo user space somma le copie solo quando legge le statistiche. È interessante notare che la pipeline modular usa già mappe per-CPU per lo scratch delle attivazioni intermedie, quindi il pattern è già presente nel codice; applicarlo anche ai contatori di tutte e tre le pipeline è coerente e a basso rischio.

Il quarto è il modo in cui viene eseguito il redirect finale. Tutte le pipeline usano `bpf_redirect(ifindex, 0)`, che accetta direttamente l’indice dell’interfaccia di uscita. Esiste però una primitiva alternativa, `bpf_redirect_map()`, che indirizza il pacchetto verso una voce di una mappa di tipo `DEVMAP`: oltre a essere leggermente più veloce nel caso singolo, questa variante abilita l’*invio in blocco* dei pacchetti rediretti, che vengono accumulati e trasmessi tutti insieme alla fine del ciclo di polling della scheda di rete, invece che uno alla volta. Su traffico ad alto rate questa differenza è tutt’altro che trascurabile, ed è la ragione per cui la documentazione di XDP raccomanda il path basato su `DEVMAP` per i data plane di produzione.

Il primo di questi interventi – la conversione di `mac_table` da `BPF_HASH` a `BPF_ARRAY` – è stato effettivamente applicato in tutte e tre le pipeline, insieme alla necessaria modifica della logica di rilevamento di una classe non provvista (dato che

una `BPF_ARRAY` non restituisce mai un puntatore nullo, la condizione “classe non configurata” è ora rilevata da un `ifindex` pari a zero, valore che non è mai un’interfaccia di uscita valida). I numeri della Tabella 2.2 sono stati raccolti prima di questa modifica, e una loro rimisurazione mostrerebbe un piccolo ulteriore miglioramento sul percorso di redirect di tutte e tre le pipeline. Gli altri tre interventi – l’eliminazione del doppio parsing, i contatori per-CPU e il redirect basato su `DEVMAP` – non sono invece stati applicati: cambiarli richiederebbe una rimisurazione completa sull’hardware di validazione, e alcuni (in particolare la contesa dei contatori condivisi) hanno un effetto visibile solo su hardware multi-core reale, non nell’ambiente di laboratorio virtualizzato usato. Restano comunque tutti indipendenti dalla scelta tra `hardcoded`, `template` e `modular` – riguardano il framework comune, non l’inferenza – e rappresentano la direzione più immediata di ottimizzazione per una futura versione del datapath.

2.9 Il costo della flessibilità e la scelta della pipeline vincente

Il design space si riassume in una frase: non esiste un’implementazione unica per portare un modello neurale nel data plane, esiste una scelta su quanto specializzare il codice sul singolo modello. La pipeline `hardcoded` massimizza le prestazioni a scapito della flessibilità; la `template` trova un compromesso pratico, un solo programma per un’intera famiglia di architetture aggiornabile scrivendo una mappa; la `modular` massimizza la flessibilità, rendendo dinamiche larghezza e profondità, al prezzo del numero di tail call più alto.

Proprio perché la pipeline `hardcoded` vince così nettamente sul piano prestazionale – ed è quella più vicina al caso d’uso in cui il modello è noto in anticipo, come nello scenario di routing che si sta studiando, in cui i modelli cambiano relativamente di rado – il resto di questa tesi si concentra su di essa. È la scelta che reputiamo superiore per questo tipo di applicazione, e i suoi due limiti evidenti (un input vector cablato su un solo scenario e un costo di aggiornamento molto alto) non sono difetti intrinseci della strategia di specializzazione, ma conseguenze di scelte implementative specifiche, entrambe superabili senza sacrificare le prestazioni. Il capitolo seguente racconta come.

Capitolo 3

L'evoluzione della pipeline hardcoded

La pipeline hardcoded è la scelta vincente sul piano prestazionale: aggiunge appena 14 nanosecondi al pavimento imposto dal solo framework XDP, e questo la rende la candidata più naturale per uno scenario in cui i modelli sono noti in anticipo e cambiano relativamente di rado. Proprio perché è la pipeline su cui puntare, ha senso investire nel superare i suoi due limiti più evidenti, che il relatore ha segnalato esplicitamente durante una revisione del lavoro: l'input vector era cablato su un solo scenario di rete, e ogni aggiornamento del modello comportava un costo di ricompilazione molto alto. Questo capitolo racconta come sono stati affrontati entrambi i problemi, e un terzo filone di lavoro emerso naturalmente dal secondo: capire se, a parità di budget di pesi, convenga rendere il modello più largo o più profondo.

Il vincolo tenuto fermo per tutto il lavoro, e che vale la pena esplicitare subito, è che per i modelli noti a priori – il caso d'uso che stiamo studiando – a parità di tipo di feature la dimensione è fissa all'interno della stessa rete: è una proprietà della *topologia*, condivisa da tutti i nodi, non un parametro che ciascun modello sceglie liberamente. I modelli scelgono solo *quali* tipi di feature usare, non quanto grande sia ciascun tipo. Questa distinzione, apparentemente pedante, è la chiave che rende risolvibile in modo pulito il primo limite.

3.1 Un input vector generico, senza perdere prestazioni

Nella versione originale l'input vector era fisso nella forma 65-x-x-7: sessantacinque feature in ingresso, sempre nello stesso ordine (`link_state`, `iface`, `ttl`, `node`), pensate per lo scenario di routing basato su FRR. Un modello pensato per uno scenario diverso – con feature diverse, o con un numero diverso di feature dello stesso tipo – semplicemente non poteva essere caricato senza riscrivere a mano il generatore di codice.

La soluzione è stata introdurre un *catalogo* dei tipi di feature che il generatore sa costruire, e un *descrittore* per-modello che elenca, in ordine, quali tipi quel modello usa. Il catalogo distingue tre categorie (*kind*) di feature: *scalar*, un singolo valore letto dal pacchetto in transito, come il TTL; *dense_vector*, un piccolo vettore letto in

un colpo solo da una mappa BPF condivisa, come `link_state` o una nuova feature `queue_occupancy` aggiunta in questa fase; e *onehot*, esattamente un indice attivo su un range di dimensione nota, come `iface` o `node`. Un modello dichiara il proprio descrittore come una semplice lista ordinata di tipi, mentre la *dimensione* di ciascun tipo – quante interfacce ha `link_state`, quanti nodi ha `node` – viene letta da un file separato condiviso da tutti i nodi della rete, il `topology_config`. Le eventuali chiavi topologiche presenti nel file del singolo modello vengono deliberatamente ignorate, perché la topologia è una proprietà della rete, non del modello.

Listing 3.1. Descrittore per-modello: solo i TIPI, non le dimensioni

```

1 features = ["link_state", "queue_occupancy", "ttl"]
2 hidden_dims = [4, 4]
3 n_out = 7
4 # N_IN = somma delle dimensioni dei tipi elencati,
5 #       lette dal topology_config della rete

```

Il generatore, per ciascun tipo nel descrittore, produce un frammento di codice diverso a seconda del kind: per una feature scalare legge il valore dal pacchetto; per una feature dense emette un'unica lettura di mappa seguita da un certo numero di letture dirette dal valore restituito; per una feature one-hot emette il blocco `switch` descritto nel capitolo precedente. Qualunque combinazione di tipi un modello dichiara, il codice generato costruisce esattamente quell'input vector, feature per feature, mantenendo la stessa filosofia di inferenza completamente srotolata che caratterizza la pipeline hardcoded. Per non rompere nulla di quanto già funzionava, se un modello non dichiara affatto un descrittore il generatore ricade sul comportamento storico – [`link_state`, `iface`, `ttl`, `node`] – producendo un programma byte-identico a prima: questa proprietà è stata verificata confrontando riga per riga l'output del nuovo generatore con quello del vecchio a parità di pesi, trovando solo differenze cosmetiche nell'ordine dei termini della somma, mai una differenza sostanziale.

Un rischio concreto di questo sistema più flessibile è che un modello venga addestrato assumendo una certa dimensione dell'input e venga poi caricato su un nodo la cui topologia ne produce una diversa. Prima di introdurre un controllo esplicito, questo disallineamento passava completamente inosservato: il programma veniva generato e caricato senza errori, calcolando semplicemente un'inferenza sbagliata, perché i pesi allenati per un layout venivano applicati a un input costruito con un layout diverso. È stato quindi aggiunto un controllo bloccante, eseguito prima ancora di generare una riga di codice: si legge la dimensione dell'input attesa dal checkpoint del modello – la forma del primo strato lineare, cioè quella per cui il modello è stato effettivamente addestrato – e la si confronta con la dimensione che la topologia produrrebbe per quel descrittore; se le due cifre non coincidono, viene sollevata un'eccezione esplicita. Un errore netto è decisamente preferibile a un modello che gira silenziosamente su un input incompatibile e sceglie next-hop sbagliati.

Rendere l'input vector generico ha anche offerto l'occasione per un'ottimizzazione dell'hot path che vale per tutte e tre le pipeline. Nella prima versione, ogni singolo valore di una feature dense veniva letto con una chiamata separata a `bpf_map_lookup_elem()`: sei chiamate per `link_state`, quattro per `queue_occupancy`,

fino a dieci helper call per pacchetto solo per le feature dense. Dato che tutti i valori di una stessa feature vivono comunque in un'unica entry della mappa, basta ridefinire la mappa in modo che il suo valore sia una struttura contenente l'intero vettore, e leggerla con un'unica `lookup` su chiave fissa, seguita da letture dirette sul puntatore restituito – letture che, a differenza delle helper call, non hanno costo aggiuntivo per il verifier.

Listing 3.2. Da N lookup a 1: mappa struct-valued

```

1 struct ls_vec { __u32 v[6]; };
2 BPF_ARRAY(link_state, struct ls_vec, 1);
3
4 int z = 0;
5 struct ls_vec *p = link_state.lookup(&z);
6 if (p) { ls0 = p->v[0]; ls1 = p->v[1]; /* ... */ }
```

Applicata a tutte e tre le pipeline senza toccare l'architettura a dispatcher più tail call, questa modifica riduce di cinque le letture di mappa per pacchetto della pipeline hardcoded, a inferenza invariata (stessa classe scelta, confrontata con il riferimento Python su tutta la suite di test). È il motivo per cui, come si vedrà nei confronti storici, il conteggio di letture di mappa è *diminuito* nel tempo pur avendo aggiunto flessibilità.

3.2 AOT-literal: separare la compilazione dal deploy

Il secondo limite – il costo di ricompilazione – è più sottile da risolvere, perché è intrinseco alla natura della pipeline hardcoded: se i pesi sono letterali nel codice, un cambio di pesi richiede necessariamente una nuova compilazione. Non si tratta quindi di eliminare la compilazione, ma di capire se sia possibile spostarla *fuori dal percorso critico*.

La pipeline hardcoded, nella sua forma originale, usa BCC (BPF Compiler Collection): una libreria che include al proprio interno LLVM/clang e compila il sorgente C a runtime, ogni volta che il programma viene caricato, direttamente sul nodo che farà da datapath. Il costo misurato di questa compilazione a runtime, isolando il solo tempo di `clang`, è di circa 1660 millisecondi per modello: pochissimo se il modello cambia raramente, ma un costo enorme se lo si moltiplica per molti nodi che devono ricevere lo stesso aggiornamento, o se si vuole un vero hot-swap del modello sul datapath vivo.

L'osservazione chiave è che il numero di compilazioni non deve cambiare – resta una per modello, dato che i pesi restano letterali – ma *dove e quando* questa compilazione avviene sì. È stato quindi costruito un percorso alternativo, che chiamiamo AOT-literal (Ahead-Of-Time), che genera lo stesso tipo di sorgente C con i pesi come letterali, ma nel dialetto `libbpf` invece che nel dialetto BCC, e lo compila con `clang offline`, su una build box separata dal nodo datapath, producendo un file oggetto `.o` già pronto. Il nodo datapath, a quel punto, deve solo aprire e caricare questo file già compilato – un'operazione ordini di grandezza più leggera del compilare da zero. La Figura 3.1 illustra questa separazione.

I numeri confermano l'intuizione: la build offline costa, a seconda della run, tra i 90 e i 555 millisecondi di `clang`, pagati una sola volta sulla build box, fuori dal

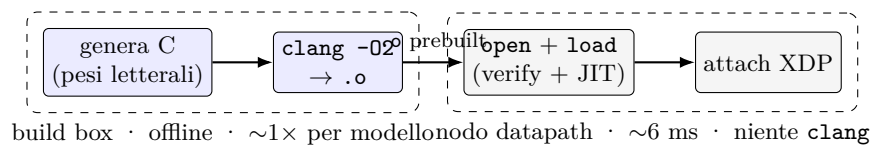


Figura 3.1. AOT-literal: la compilazione `clang` (costosa) resta una per modello ma è spostata offline sulla build box; sul nodo datapath resta solo il caricamento del file oggetto già compilato, che costa pochi millisecondi contro i ~1660 della compilazione BCC a runtime.

percorso critico. Sul nodo datapath, aprire il file oggetto e caricarlo nel kernel (verifica più JIT) costa, nell’ultima misura, circa 6 millisecondi in totale – contro i 1660 della compilazione BCC a runtime. Le prestazioni per pacchetto restano identiche a quelle della hardcoded via BCC, perché l’architettura generata (dispatcher più tail call più doppio parsing) è la stessa e i pesi restano letterali, con lo stesso beneficio di strength reduction: l’ultima misura, con la stessa metodologia usata per la tabella comparativa, dà 1010 istruzioni totali (28 di dispatcher più 982 del modello), 42 nanosecondi per pacchetto e 23.8 milioni di pacchetti al secondo. In altre parole, si ottiene lo stesso datapath, con lo stesso costo per pacchetto, ma con il costo di aggiornamento sul nodo abbattuto di quasi tre ordini di grandezza.

Nella sua prima versione il generatore AOT-literal copriva solo il descrittore di feature storico: il lavoro sul catalogo di feature descritto sopra era stato implementato solo nel dialetto BCC. Parte del lavoro è quindi consistita nel portare i tre generatori per kind di feature (scalar, dense, one-hot) anche nel dialetto `libbpf`, così che qualunque descrittore accettato dal percorso BCC sia ora compilabile anche in AOT. La differenza tra i due dialetti è puramente sintattica – funzioni “nude” come `m.lookup(&k)` in BCC contro dichiarazioni `struct {...} SEC(".maps")` e `bpf_map_lookup_elem(&m, &k)` in `libbpf` – non concettuale.

Su richiesta esplicita del relatore, questo percorso non è rimasto un’alternativa opzionale ma è diventato l’*unico* meccanismo con cui la pipeline hardcoded viene distribuita e attaccata: AOT-literal sostituisce completamente BCC come backend di deploy. Concretamente, questo ha richiesto di risolvere il vincolo che inizialmente aveva consigliato di tenere entrambi i percorsi: i nodi del laboratorio Kathara non hanno né `clang` né `libbpf` installati, mentre BCC incorpora LLVM/clang come libreria, senza bisogno di questi strumenti. Il percorso AOT non ha mai avuto bisogno di eseguire `clang` sul nodo – la compilazione avviene offline sulla build box e sul nodo arriva solo il file oggetto – ma il piccolo loader in C che apre quel file e lo carica nel kernel era collegato dinamicamente contro `libbpf`, e quindi non sarebbe partito sul nodo per l’assenza della libreria condivisa. La correzione è stata collegare staticamente il loader contro `libbpf` e le sue dipendenze (`libelf`, `zlib` e, su distribuzioni recenti, `libzstd` e `liblzma`), lasciando dinamico solo il collegamento alla libreria C standard, ragionevolmente presente su qualunque sistema Linux. Il risultato è un unico eseguibile autosufficiente: compilato una volta su una build box attrezzata, viene poi eseguito su un numero qualunque di nodi datapath privi di `clang`, `libbpf` e di qualunque libreria di sviluppo eBPF – esattamente i nodi del laboratorio usato. Con questa correzione il deploy della pipeline hardcoded è oggi interamente basato su AOT-literal, e il vecchio percorso BCC per il deploy live non

viene più invocato. L'unica eccezione, volutamente conservata, è l'infrastruttura di test, che continua a usare BCC per compilare e verificare la correttezza dell'inferenza offline: quella suite non gira mai sul nodo datapath in produzione, ma su una macchina di sviluppo dove BCC è disponibile, ed è una questione del tutto distinta dal modo in cui il modello viene realmente distribuito.

3.3 Profondità variabile e studio larghezza contro profondità

Un limite meno visibile, ma altrettanto rigido, era che sia il generatore BCC sia quello AOT assumevano sempre esattamente due hidden layer: le variabili si chiamavano letteralmente `n_h1` e `n_h2`, e tutta la logica di generazione – lo slicing dei pesi, l'emissione dei layer – era scritta esplicitamente per due strati. Entrambi i generatori (e, in modo speculare, il riferimento Python usato dalla suite di test) sono stati generalizzati in modo che il numero di hidden layer sia una sequenza di lunghezza qualsiasi: due come prima, ma anche uno solo, quattro, o nessuno (un modello puramente lineare). Il layout dei pesi diventa generico per-layer – per ogni coppia di strati consecutivi, un blocco di $n_{prev} \times n_{cur}$ pesi seguito da n_{cur} bias – e si riduce esattamente al vecchio schema quando i layer sono due. La proprietà è stata verificata sia confrontando byte per byte il codice generato per il caso a due strati con quello prodotto dal generatore precedente (identico a meno di righe vuote), sia confrontando numericamente il nuovo riferimento Python generalizzato con la vecchia implementazione esplicita a due strati, su pesi e input casuali.

Questa generalizzazione, di per sé un dettaglio, ha reso possibile l'esperimento più originale di questa parte del lavoro: a parità di numero di pesi – e quindi, grosso modo, a parità di capacità del modello – conviene concentrare quei pesi in pochi strati larghi, o distribuirli su molti strati stretti? La domanda ha un risvolto molto pratico per la pipeline hardcoded, dato che l'intera inferenza vive dentro un'unica funzione C srotolata, soggetta al limite di 512 byte di stack che il verifier impone su ogni funzione.

Per rispondere è stato costruito uno script di benchmark dedicato che confronta, a tre livelli di budget di pesi crescente (circa 300, 1200 e 4700 pesi), una forma “larga” (un solo hidden layer) contro due forme “profonde” (quattro e otto hidden layer), dichiarando sempre esplicitamente lo scarto reale tra i budget delle diverse forme invece di darlo per scontato come uguale. Per evitare che la conclusione dipendesse in modo nascosto dal particolare descrittore di feature usato di default – che contiene una feature one-hot costosa, `node`, con 52 valori – lo stesso esperimento è stato ripetuto su quattro descrittori diversi: quello storico con due feature one-hot, uno senza nessuna, uno con una sola one-hot piccola e uno con una sola one-hot grande. Le misure di latenza usano la stessa statistica del minimo su più trial indipendenti discussa nella Sezione 2.7, resa qui ancora più necessaria dal rumore. Un secondo problema, distinto, ha riguardato le configurazioni più estreme: alcune sfiorano il limite di stack, e quando questo accade il backend LLVM di BCC termina con un errore fatale a livello di processo, non con un'eccezione catturabile. Poiché tentare di prevedere a priori quali forme avrebbero sfiorato lo stack si è rivelato inaffidabile – il compilatore a volte riesce a riutilizzare lo spazio di stack tra variabili

con vita non sovrapposta e a volte no, in modo non prevedibile – la soluzione robusta è stata eseguire ciascuna configurazione in un sottoprocesso isolato: se muore per un errore fatale, viene segnata come fallita e l'esperimento prosegue.

Tabella 3.1. Larghezza contro profondità a parità di budget di pesi (minimo su 15 trial, descrittore storico).

Configurazione	Pesi	Latenza minima (ns/pkt)	Note
Baseline / largo, 1×4	$\sim 300\text{--}320$	44–56	riferimento
Profondo, 8×3	~ 310	57–76	overhead contenuto
Largo, 1×16	~ 1175	103–111	sempre il più veloce
Profondo, 4×11	~ 1206	203–236	circa $2 \times$ più lento
Profondo, 8×9	~ 1294	269–301	$2.5\text{--}3 \times$ più lento
Qualunque forma	$\sim 4700\text{--}4780$	—	sempre fallito

Il risultato, riassunto nella Tabella 3.1, è netto e coerente su tutti e quattro i descrittori: a parità di budget di pesi, allargare un modello è sistematicamente più veloce che approfondirlo, e la differenza cresce con il budget, arrivando a un fattore due o tre al livello intermedio. Ogni strato nascosto aggiuntivo introduce un costo fisso, dell'ordine di 15–40 nanosecondi, dovuto alla transizione tra uno strato e il successivo e all'attivazione ReLU, indipendente dalla composizione di feature del descrittore. Il secondo risultato, forse più interessante dal punto di vista architetturale, è che al livello di budget più alto *ogni* configurazione – larga o profonda – sfiora il limite di stack e fallisce in compilazione, indipendentemente dal descrittore. Esiste cioè un tetto di capacità assoluto per l'architettura hardcoded così com'è costruita (tutta l'inferenza srotolata in un'unica funzione, pesi come letterali): non è questione di scegliere bene tra larghezza e profondità, è un limite strutturale del design, superabile solo spostando gli array più grandi in una mappa PERCPU_ARRAY – suggerimento che, non a caso, compare testualmente nel messaggio di errore del compilatore – invece di continuare a redistribuire lo stesso numero di pesi.

3.4 Nota conclusiva sull'evoluzione

Le tre linee di lavoro descritte in questo capitolo condividono un filo comune: nessuna sacrifica il vantaggio prestazionale che rende la pipeline hardcoded la scelta migliore per lo scenario studiato. L'input vector generico rende il modello portabile su scenari diversi mantenendo il codice byte-identico nel caso storico; l'AOT-literal abbassa il costo di aggiornamento di quasi tre ordini di grandezza lasciando invariato il costo per pacchetto; la profondità variabile apre lo spazio delle architetture esplorabili e, insieme allo studio larghezza-contro-profondità, individua sia una regola pratica di progettazione (allargare batte approfondire) sia il limite strutturale oltre il quale l'attuale design non può crescere senza un ripensamento. È in questo senso che la pipeline hardcoded, pur essendo la più specializzata e apparentemente la più rigida delle tre, si è rivelata la più promettente da far evolvere.

Bibliografia

- [1] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller. *The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel*. Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT), 2018.
- [2] G. Heiser et al. *Benchmarking Crimes: An Emerging Threat in Systems Security*. arXiv:1801.02381, 2018.
- [3] Z. Jones. *Performance Analysis of XDP Programs*. USENIX LISA21, 2021.
- [4] *In-Kernel CNN Inference for Edge Devices: An eBPF-Based Approach to Low-Latency and Resource-Efficient Processing*. Springer, 2025.
- [5] *Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree*. Computer Networks, 2024.